

JSON desde cero

Representar, validar y consultar datos

PROGRAMA	Programa de especialización
MÓDULO	UD2 · JSON
FECHA	Septiembre de 2026

Objetivo general

Aprender a utilizar JSON para representar, estructurar, validar y consultar datos, partiendo desde cero y llegando al uso de JSON en APIs.

El aprendizaje será práctico y progresivo, utilizando Visual Studio Code y ejemplos relacionados con una tienda online.

Bloque 1 · ¿Qué es JSON?

¿Qué problema resuelve JSON?

- JSON es un formato para representar e intercambiar datos.
- Es muy utilizado para comunicar información entre aplicaciones.
- Es fácil de leer tanto para personas como para máquinas.

IDEA CLAVE

JSON = datos estructurados

EJEMPLO

```
{  
  "nombre": "Ratón",  
  "precio": 19.99  
}
```

Bloque 2 · Estructura básica de JSON

Objetos {}

Un objeto contiene propiedades y valores.

```
{  
  "nombre": "Juan",  
  "edad": 20,  
  "ciudad": "Valencia"  
}
```

Conceptos:

Elemento	Significado
{ }	Objeto
"nombre"	Propiedad
"Juan"	Valor
:	Separa propiedad y valor
,	Separa propiedades

REGLA BÁSICA

"propiedad": valor

Bloque 3 · Tipos de datos

Tipos de datos en JSON

JSON permite trabajar con los siguientes tipos:

Tipo	Ejemplo
String	"Juan"
Number	25
Boolean	true / false
Null	null
Object	{ }
Array	[]

CONCEPTO IMPORTANTE

"edad": 20 no es lo mismo que "edad": "20". El primero es un **número**. El segundo es **texto**.

Bloque 4 · Arrays

Arrays []

Un array representa una lista de valores.

```
{  
  "aficiones": [  
    "fútbol",  
    "música",  
    "videojuegos"  
  ]  
}
```

IDEA CLAVE

{ } → objeto · [] → lista / array

Un array puede contener:

- textos
- números
- booleanos
- objetos
- otros arrays

Bloque 5 · Objetos anidados

Objetos dentro de objetos

Los objetos pueden contener otros objetos.

```
{  
  "nombre": "Juan",  
  "direccion": {  
    "calle": "Colón",  
    "ciudad": "Valencia",  
    "codigoPostal": "46001"  
  }  
}
```

CONCEPTO

Los datos pueden estar anidados formando una **estructura jerárquica**.

Bloque 6 - Objetos dentro de arrays

Arrays de objetos

Esta estructura es muy habitual en JSON.

```
{
  "productos": [
    {
      "nombre": "Ratón",
      "precio": 19.99
    },
    {
      "nombre": "Teclado",
      "precio": 29.99
    }
  ]
}
```

INTERPRETACIÓN

```
productos
├── producto
│   ├── nombre
│   └── precio
└── producto
    ├── nombre
    └── precio
```

Bloque 7 - XML vs JSON

Comparación entre XML y JSON

Los dos permiten representar los mismos datos utilizando estructuras diferentes.

XML

```
<producto>
  <nombre>Ratón</nombre>
  <precio>19.99</precio>
  <stock>20</stock>
  <activo>true</activo>
</producto>
```

JSON

```
{  
  "nombre": "Ratón",  
  "precio": 19.99,  
  "stock": 20,  
  "activo": true  
}
```

Diferencias principales

XML	JSON
Etiquetas	Propiedades
Atributos	No tiene atributos como XML
XSD	JSON Schema
XPath	JSONPath
Más verboso	Más compacto
Muy usado en documentos estructurados	Muy usado en aplicaciones y APIs

IDEA CLAVE

Los datos pueden ser los mismos; cambia la forma de representarlos.

Bloque 8 · Sintaxis y errores

Aprender a detectar errores

JSON tiene unas reglas de sintaxis estrictas.

Error 1 — Coma final

INCORRECTO

```
{  
  "nombre": "Juan",  
  "edad": 20,  
}
```

Error 2 — Propiedad sin comillas

INCORRECTO

```
{  
  nombre: "Juan"  
}
```

CORRECTO

```
{  
  "nombre": "Juan"  
}
```

Error 3 — Boolean incorrecto

INCORRECTO

```
{  
  "activo": True  
}
```

CORRECTO

```
{  
  "activo": true  
}
```

CONCEPTO IMPORTANTE

Un JSON puede ser correcto sintácticamente e incorrecto según las reglas que queremos imponer. Esta diferencia será importante al estudiar JSON Schema.

Bloque 9 · JSON en Visual Studio Code

Trabajar con JSON en VS Code

Crear un archivo llamado `tienda_online.json`. VS Code permite:

- Detectar errores de sintaxis.
- Colorear la estructura.
- Autocompletar.
- Formatear JSON.
- Trabajar con archivos `.json`.

OBJETIVO

Aprender JSON directamente sobre archivos reales, sin necesidad de programación.

Bloque 10 - Proyecto: tienda online

Construir nuestro propio JSON

Crearemos progresivamente el archivo `tienda_online.json`.

Nivel 1 — Información básica

```
{  
  "nombre": "Tienda Online"  
}
```

Nivel 2 — Más información

```
{  
  "nombre": "Tienda Online",  
  "email": "tienda@email.com",  
  "telefono": "123456789"  
}
```

Nivel 3 — Dirección

```
{  
  "nombre": "Tienda Online",  
  "direccion": {  
    "calle": "Colón",  
    "ciudad": "Valencia",  
    "codigoPostal": "46001"  
  }  
}
```

Nivel 4 — Productos

```
{  
  "nombre": "Tienda Online",  
  "productos": [  
    {  
      "id": "P001",  
      "nombre": "Ratón",  
      "precio": 19.99,  
      "stock": 20  
    },  
    {  
      "id": "P002",  
      "nombre": "Teclado",  
      "precio": 29.99,  
      "stock": 15  
    }  
  ]  
}
```

Niveles siguientes

Nivel	Objetivo
05	Añadir clientes.
06	Añadir pedidos.
07	Relacionar pedidos con clientes y productos mediante identificadores.

Bloque 11 - JSON Schema

¿Cómo establecemos reglas para JSON?

En XML utilizamos **XSD**. En JSON utilizamos **JSON Schema**.

XML → XSD
JSON → JSON Schema

IDEA CLAVE

JSON contiene los **datos**. JSON Schema establece las **reglas**.

Bloque 12 - Tipos en JSON Schema

Definir tipos

```
{
  "type": "object",
  "properties": {
    "nombre": {
      "type": "string"
    },
    "precio": {
      "type": "number"
    }
  }
}
```

Esto establece que:

- **nombre** debe ser texto.
- **precio** debe ser un número.

Relación con XSD

Lenguaje	Definición de tipo
XSD	<code>type="xs:string"</code>
JSON Schema	<code>"type": "string"</code>

Bloque 13 - Validación

Validar un JSON

JSON CORRECTO

```
{
  "nombre": "Ratón",
  "precio": 19.99
}
```

JSON INCORRECTO SEGÚN EL SCHEMA

```
{
  "nombre": "Ratón",
  "precio": "diecinueve euros"
}
```

IDEA FUNDAMENTAL

JSON válido sintácticamente $\neq$ JSON válido según un JSON Schema.

Bloque 14 - JSONPath

Buscar información dentro de JSON

En XML hemos utilizado **XPath**. Para JSON podemos utilizar **JSONPath**.

```
{
  "productos": [
    {
      "nombre": "Ratón",
      "precio": 19.99
    },
    {
      "nombre": "Teclado",
      "precio": 29.99
    }
  ]
}
```

Objetivo: aprender a localizar:

- Todos los productos.
- Los nombres.
- Los precios.
- Un producto concreto.
- Datos que cumplen determinadas condiciones.

Bloque 15 · JSON y APIs

¿Dónde encontramos JSON en el mundo real?

JSON es muy utilizado en APIs. Una aplicación puede solicitar información a otra aplicación y recibir los datos en JSON.

```
Aplicación
  ↓
  API
  ↓
  JSON
  ↓
Aplicación
```

EJEMPLO

```
{
  "id": 123,
  "nombre": "Juan",
  "email": "juan@email.com"
}
```

Bloque 16 - REST y JSON

JSON en APIs REST

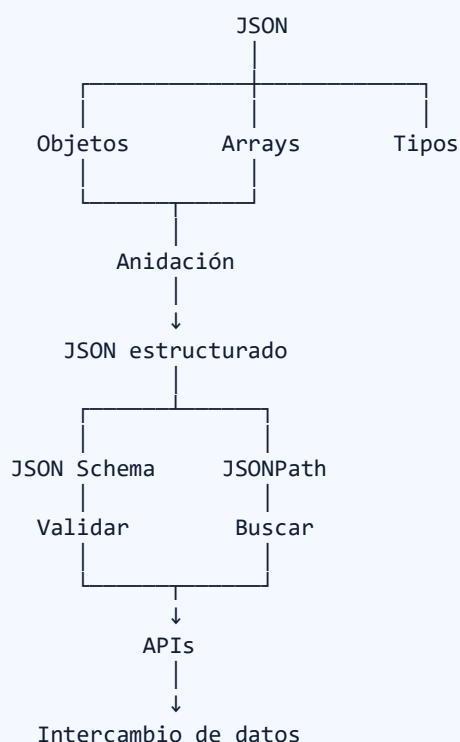
Introducción a los métodos HTTP y su relación con JSON:

Método	Acción	Uso con JSON
GET	Obtener datos	GET + JSON
POST	Crear datos	POST + JSON
PUT	Modificar datos	PUT + JSON
PATCH	Modificar parcialmente	PATCH + JSON
DELETE	Eliminar datos	DELETE + JSON

NOTA

No se requiere programación inicialmente. El objetivo es comprender cómo se intercambian datos.

Mapa final de conceptos



Objetivo final

Al terminar el recorrido, el alumno debe ser capaz de:

1. Explicar qué es JSON.
2. Crear documentos JSON desde cero.
3. Identificar objetos, arrays y tipos de datos.
4. Crear estructuras JSON anidadas.
5. Detectar errores de sintaxis.
6. Diferenciar XML y JSON.
7. Crear un JSON Schema.
8. Validar documentos JSON.
9. Realizar consultas mediante JSONPath.
10. Entender cómo JSON se utiliza en APIs.
11. Diseñar estructuras JSON para un problema real.
12. Comprender el flujo completo:

DATOS → JSON → VALIDACIÓN → CONSULTA → API